

Problem Set Resources and Guidelines

9/23/26

Contents

Submitting your problem set	1
Questions and class discussion	1
How the problem sets and autograder work	2
(1) Problem sets are R scripts (.R files)	2
(2) Your answers must be assigned to specifically named objects	2
(3) Make sure your whole script runs, top to bottom, without errors	2
(4) Don't alter provided code	2
(5) The data loads from the internet	2
(6) You will not see an automated score	2
Working with directories and files	3
(1) Re-watch these videos on relative and absolute paths	3
(2) <code>getwd()</code> view your working directory	3
(3) <code>list.files()</code> list the files in a folder	3
(4) <code>setwd()</code> set your working directory	3

Submitting your problem set

1. Rename your script using the convention `lastname_firstname_problemset#`, for example `jaquette_ozan_problemset1.R`.
2. Go to the course **BruinLearn** page and open the assignment for that problem set.
3. Upload your .R script to that assignment.

Questions and class discussion

We encourage you to post questions about the problem sets and lectures as a GitHub issue. Post issues here: [rclass1 student issues \(Fall 2026\)](#).

Each problem set also asks you to post one issue and reply to at least one other student's issue for credit.

Issue guidelines

- Use a clear title describing your issue (e.g. "PS1: file directory question").
- Provide enough background to understand the problem: a description, your code, any relevant links, etc.

How the problem sets and autograder work

Please read this section carefully before starting Problem Set 1.

(1) Problem sets are R scripts (.R files)

Each problem set is a single R script (e.g. `problemset1.R`) that you download from the class website. You will open it in RStudio, write your code directly in the script, and submit the .R file.

(2) Your answers must be assigned to specifically named objects

Throughout each problem set, you will be asked to assign your answer to an object with an **exact name**, for example:

```
q1_1 <- c(76, 7248, 2912)
q2_4 <- "yes"
```

- **The object name must match exactly.** If the question asks you to name your answer `q2_2` and you name it `q2.2`, `Q2_2`, or `answer`, the autograder will not find it and you will receive no credit (even if your answer is correct).
- **Character vs. numeric matters.** When an answer is a word, wrap it in quotes (`"yes"`, `"unitid"`). When it is a number, leave it unquoted (`5`). When an answer has multiple parts, combine them with `c()` exactly as the question shows (`c("var1", "var2")` or `c(1, 2, 3)`). The problem sets tell you which format each answer expects so follow it precisely.

(3) Make sure your whole script runs, top to bottom, without errors

Before you submit, always do this:

1. Restart R so the whole session is clean: **Session > Restart R**
2. Run the entire script from the first line to the last.
3. Confirm it runs with **no errors** and that every `qX_X` answer object shows up in your Environment.

(4) Don't alter provided code

Some questions give you code and tell you to simply run it ("all code is provided for you"). Leave that code as-is because later questions depend on it.

(5) The data loads from the internet

Each problem set loads its data directly from a web link inside the script. For example:

```
df_recruiting <- read.csv("https://raw.githubusercontent.com/anyone-can-cook/rclass1/refs/heads/master/data/recruiting/recruit_ps1.csv")
```

This means you must be **connected to the internet** when you run the problem set. If a data-loading line fails, first check your connection before assuming your code is wrong.

(6) You will not see an automated score

When you submit, you will not get an instant grade back. The TA runs the autograder, reviews the code, and enters your scores afterward.

Working with directories and files

Problem Set 1 asks you to create an R project and confirm your working directory, so it is worth getting comfortable with these functions early.

(1) Re-watch these videos on relative and absolute paths

- Absolute versus relative file paths [Youtube link](#)
- Relative paths and working directory in R [Youtube link](#)

(2) `getwd()` view your working directory

The `getwd()` function prints the filepath of your current working directory. When you open a problem set inside its R project, this should point to the problem set's folder.

```
getwd()

## [1] "/Users/jadasims/Desktop/rclass1_f26/rclass1/problem_sets"
```

(3) `list.files()` list the files in a folder

`list.files()` lists the files in your current working directory. This is useful in Problem Set 1 to confirm that your `.R` script and `.Rproj` file are where you expect them.

```
list.files()      # lists files in the current working directory
?list.files       # type this in the console to read the help file
```

You can also point it at a specific folder:

```
list.files(path = "data/ipeds")
```

(4) `setwd()` set your working directory

`setwd()` changes your working directory. In general, if you open each problem set inside its own R project, you should rarely need to call `setwd()` yourself.

```
setwd("../intro_to_r")
```

Note: when `setwd()` is run inside a code chunk, the change applies only within that chunk. For your `.R` problem set scripts, prefer working inside an R project so that paths behave predictably and **do not** hard-code an absolute path to a folder on your own computer, since that will not run on anyone else's machine (including the grader's).